

Distributed out-of-core MergeSort

Parallel and Distributed Systems: Paradigms and Models

Gemma Ragadini

Matricola: 545391

Università degli Studi di Pisa

A.A. 25/26

Indice

1	Introduction	3
2	Data model and constraints	3
2.1	Index structure	3
3	Pipeline	4
4	Design choices	4
5	Single-node implementations	5
5.1	Sequential (baseline)	5
5.2	OpenMP	5
5.3	FastFlow	5
6	Distributed MPI+OpenMP version	6
6.1	Data representation and MPI datatype	6
6.2	Algorithmic pipeline	6
6.2.1	Index build	6
6.2.2	Initial distribution	7
6.2.3	Local sorting with OpenMP	7
6.2.4	Splitter selection via sampling	7
6.2.5	Local bucketization and exchange	7
6.2.6	Local merge and verification	7
7	Experiments	7
7.1	Single node	8
7.2	Hybrid version	10
8	Cost Model	11
8.1	Index build	12
8.2	Initial distribution	12
8.3	Local sorting with OpenMP	12
8.4	Sampling and splitter extraction	12
8.5	Bucketization and exchange	12
8.6	Local k-way merge	13
8.7	Bottleneck analysis	13

1 Introduction

This work presents the design and implementation of a scalable MergeSort for a binary file containing N variable-length records, under an out-of-core constraint: the dataset may exceed the RAM available on a single node (maximum 32 GB). The algorithm constructs an in-memory index containing triples (key, offset, len). The index is sorted using different parallel implementations, while the actual records remain on disk and are copied in sorted order during the rewrite phase.

Memory and I/O considerations are discussed, speedup and efficiency metrics as well as strong and weak scaling curves are reported, and a simple cost model for the distributed version is proposed.

2 Data model and constraints

Each record is structured as follows:

- `key` (unsigned long, 8 byte) - the only field used for comparisons
- `len` (uint32_t) - length of the payload
- `payload[len]` - arbitrary blob

The out-of-core constraint implies that loading the entire file containing the records into memory is not feasible, since the total size may exceed the RAM available on a single node. The solution therefore does not sort the records “in-place”, but instead adopts a compact representation that keeps in memory only the information necessary for comparing and locating the records.

2.1 Index structure

`IndexRec` structure is defined as:

- `key`
- `offset` - byte offset of the record in the input file
- `len`

On a 64-bit architecture, the structure occupies 24 bytes (including padding for alignment). During sorting, the following are kept in memory:

- the `idx` vector of size N
- a temporary buffer of the same size for MergeSort

The memory footprint is therefore independent of the total payload size, so the 32 GB constraint is satisfied as long as the full index fits in memory.

3 Pipeline

All variants share the following phases:

1. **Dataset generation/retrieval:**
`ensure_unsorted_file(n_records, payload_max)` generates a file `data/usorted_N_P.bin` using a deterministic RNG (seed 42), or reuses it if it already exists.
2. **Build index (out-of-core):**
performs a sequential scan of the file using `mmap`; for each record only `key` and `len` are read.
3. **Index sorting (in-memory):**
One of the following implementations can be used:
 - Sequential
 - OpenMP task-based mergesort (cutoff)
 - FastFlow `ParallelFor` (block sort + iterative merge)
 - MPI + OpenMP (scatter + local sort + sampling splitters + `alltoallv` exchange + local k-way merge)
4. **Rewrite of the sorted file (out-of-core):**
`rewrite_sorted_file_mmap(in, out, idx_sorted)` creates the output file and copies the records, according to the sorted index, from the mapped input file to the output.
5. **Correctness verification:**
`check_sorted_file_mmap(out, N)` checks:
 - non-decreasing keys
 - correct number of records
 - exact EOF (no extra bytes)

4 Design choices

The adopted solution avoids loading the entire dataset into memory and instead operates on a compact representation, namely the record index, separating the phase in which the index is sorted in RAM from the phase in which the file is rewritten in sorted order. This separation enables a cleaner performance analysis, making the benefits of parallel sorting more evident as the number of threads/ranks increases.

During the index build phase, access is sequential and therefore favorable to kernel prefetching. In the rewrite phase, instead, the input is read according to offsets determined by the final sorted order, while the output is written sequentially. As a result, this phase can be significantly influenced by the behavior of the page cache and the filesystem.

The comparator sorts by `(key, offset)` rather than by key alone, in order to produce deterministic output even in the presence of duplicate keys and parallel execution.

5 Single-node implementations

5.1 Sequential (baseline)

The sequential version implements a recursive top-down MergeSort applied to the `std::vector<IndexRec>`, explicitly implemented with the same logic used in the other versions. This version represents the **baseline** against which the performance of the other implementations is compared.

The main functions are `mergesort_index_seq`, which allocates a temporary vector `tmp` of size `N`, and `mergesort_rec`, the recursive function. In the recursive case, the midpoint is computed and recursion proceeds on the two halves. Finally, through the function `merge_range`, two already sorted ranges are merged into `tmp` and the result is copied back. The function `index_less` compares the `key` field, which is the sorting criterion.

5.2 OpenMP

The OpenMP version parallelizes MergeSort using the OpenMP task-based model. The idea is to transform the MergeSort recursion tree into a set of dynamic tasks: each recursive call that splits the interval generates two tasks (left half and right half), which can be scheduled in parallel by the runtime.

Similarly to the sequential version, the entry point is `mergesort_index_openmp`, and the recursion is performed in `mergesort_rec_parallel`, invoked through:

```
1 #pragma omp parallel
2 {
3     #pragma omp single
4     mergesort_rec_parallel(...);
5 }
```

To prevent the number of tasks from growing excessively, a **cutoff** parameter is used, so that if $right - left \leq cutoff$ the sorting is performed sequentially using `std::sort`.

In the other cases, the interval is divided into two halves and two tasks are generated:

```
1     #pragma omp task shared(a,tmp)
2     mergesort_rec_parallel(... left, mid ...);
3
4     #pragma omp task shared(a,tmp)
5     mergesort_rec_parallel(... mid, right ...);
```

Before performing the merge, a barrier is enforced through `taskwait`, so that the thread waits for the completion of the child tasks, preserving correctness.

Finally, the merge phase is the same as in the sequential version (`merge_range`)

5.3 FastFlow

The FastFlow version implements parallel sorting of the index using the `ff::ParallelFor` building block. An iterative bottom-up strategy is adopted: first independent

blocks are sorted, and then parallel merges with increasing width are performed. `mergesort_index_ff` uses `cutoff` (the base block size) and `nw` (number of workers).

`idx` is divided into contiguous blocks of size `block = max(1, cutoff)`, each of which is sorted independently using `std::sort`:

```

1 pf.parallel_for(0, n, block, [&](size_t l) {
2     size_t r = min(l+block, n);
3     std::sort(idx.begin()+l, idx.begin()+r, index_less);
4 });

```

After sorting the blocks, the merge proceeds in multiple passes: source and destination arrays, `src` and `dst`, are allocated, and in each pass sorted segments of width `width` are merged, with `width` doubling at each step (`block`, `2block`, `4block`, ...). For a given `width`, the pass combines pairs of segments $[left, mid)$ and $[mid, right)$, where $mid = \min(left + width, n)$ and $right = \min(left + 2 * width, n)$. The merge of each pair is independent and therefore parallelized using:

```

1 pf.parallel_for(0, n, step, [&](size_t left) {
2     ... merge src[left:mid) and src[mid:right) into dst[left:
3     right) ...
4 });

```

6 Distributed MPI+OpenMP version

The distributed version combines MPI across nodes and OpenMP within each rank to sort the record index. The algorithm follows a range-partitioning scheme that, after a single exchange phase, produces segments that are globally ordered across ranks. The hybrid version uses OpenMP only for local computation (index sorting) and MPI only in the main thread.

6.1 Data representation and MPI datatype

The elements exchanged between processes are `IndexRec` structures (key, offset, len). To correctly transmit the structure, a custom `MPI_Datatype` is created using `MPI_Type_create_struct`, with `MPI_Get_address` used to compute the actual memory displacements. This choice avoids making assumptions about padding or alignment within the struct.

6.2 Algorithmic pipeline

Let P be the number of MPI processes and N the number of records.

6.2.1 Index build

Rank 0:

- generates or reuses the dataset
- builds the global index `global_idx` (a vector of `IndexRec`)

This phase is sequential and constitutes a potential bottleneck. The path of the input file is then broadcast to all ranks using `MPI_Bcast`.

6.2.2 Initial distribution

Rank 0 computes `counts[]` and `displs[]` to divide the index into P contiguous chunks of size approximately N/P . The chunks are sent to the ranks using `MPI_Scatterv`. This initial distribution is based on the position in the index and not on key ranges.

6.2.3 Local sorting with OpenMP

Each rank sorts its own chunk `local_idx` using the OpenMP task-based version; the number of threads is set through `omp_set_num_threads`. Local correctness is verified with `is_sorted_by_key`, and an `MPI_Allreduce(MIN)` ensures that all ranks have produced a sorted chunk.

6.2.4 Splitter selection via sampling

To obtain a global distribution by key ranges, each rank extracts a sample of keys from its sorted chunk at equally spaced positions. The samples are collected on rank 0 using `MPI_Gatherv`, sorted, and $P-1$ splitters are selected as quantiles; these are then broadcast to all ranks.

6.2.5 Local bucketization and exchange

Since `local_idx` is sorted, bucketization is efficient: for each splitter the boundary is found using `std::lower_bound`, and each bucket corresponds to a contiguous segment of the local array.

Rank r therefore builds P buckets: bucket j contains only keys that belong to the range assigned to rank j . The buckets are exchanged with a single `MPI_Alltoallv`. After this step, each rank owns only the elements belonging to its global key interval, received as the concatenation of P already sorted segments (one from each source rank).

6.2.6 Local merge and verification

The received segments are already individually sorted; to obtain a single sorted vector per rank, a **k-way merge** is performed using a min-heap (`priority_queue` with an inverted comparator).

Local correctness is verified using `is_sorted_by_key`, and an additional boundary check is performed: $\max(\text{rank } r) \leq \min(\text{rank } r + 1)$

7 Experiments

The experiments were conducted on the **spmcluster** cluster. Each node provides 32 CPU cores, which represent the maximum degree of parallelism available for the single-node tests. In the distributed tests, up to 8 nodes were used, always maintaining 32 cores per node.

To analyze the performance of the algorithm, during the execution of the tests I recorded separately the execution times associated with the main phases of the program. In particular, I measured and logged several specific timing components, including `build_time` (time required to construct the data structures), `kernel_time`

(build time + sorting time + merging time), `sort_time`, and `part_time` (time for the data partitioning operations among the different processes or threads).

Unless otherwise specified in the plots, “time” corresponds to `kernel_time`. For the hybrid version of the algorithm, I produced a **time breakdown** plot for a representative configuration, using 4 nodes in the strong scaling experiment and 4 ranks per node in the weak scaling experiment, in order to observe how the different time components scale.

7.1 Single node

For the single-node experiments, the **Sequential**, **OpenMP**, and **FastFlow** versions of the algorithm are compared. For each configuration, the resource budget was fixed to $cpus - per - task = 32$, and the number of threads was varied in the sequence 1, 2, 4, 8, 16, 24, 30, 32. Each configuration was executed twice and the results were saved in a CSV file. Two scenarios were considered.

- **Scenario A:** $N = 10^8$ records, $payload_max = 16$ bytes
- **Scenario B:** $N = 10^6$ records, $payload_max = 256$ bytes

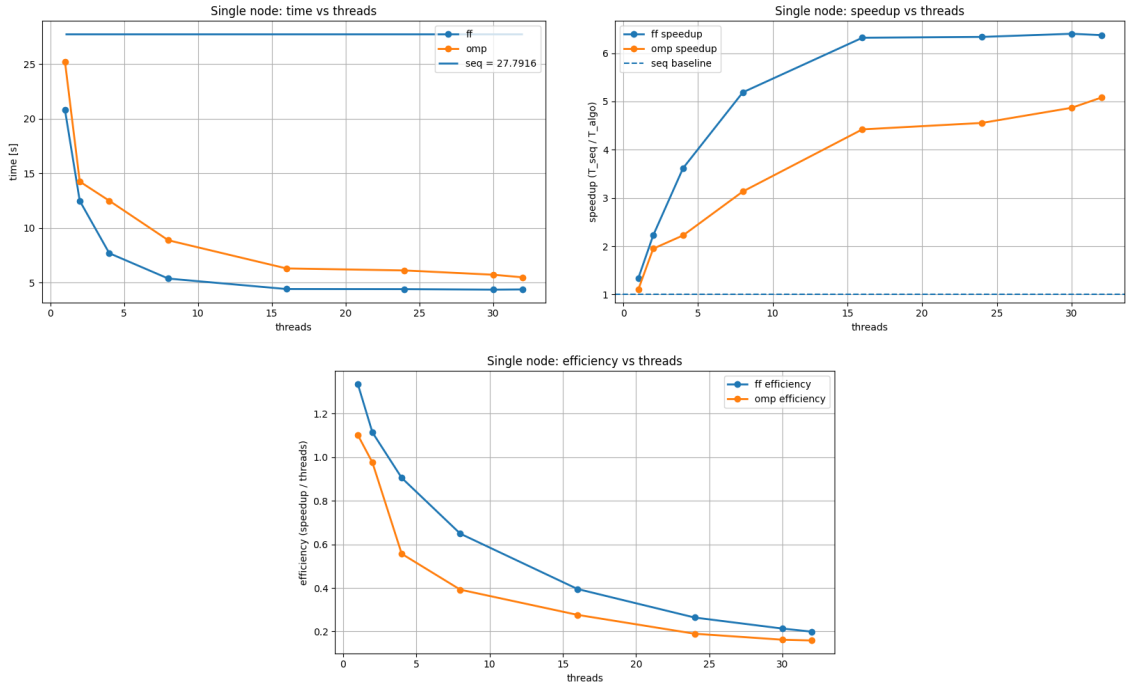


Figure 1: Single node test scenario A

The results show that both parallel implementations achieve significant speedup compared to the sequential baseline.

In Scenario A (large number of records and small payload), the execution time is dominated by the sorting of the index. In this case, the FastFlow implementation achieves the best scalability, reaching close to linear speedup up to around 16–24 threads, after which the improvement becomes marginal due to memory bandwidth limitations and synchronization overheads. The OpenMP task-based implementation also scales well, but shows slightly lower efficiency at high thread counts. This

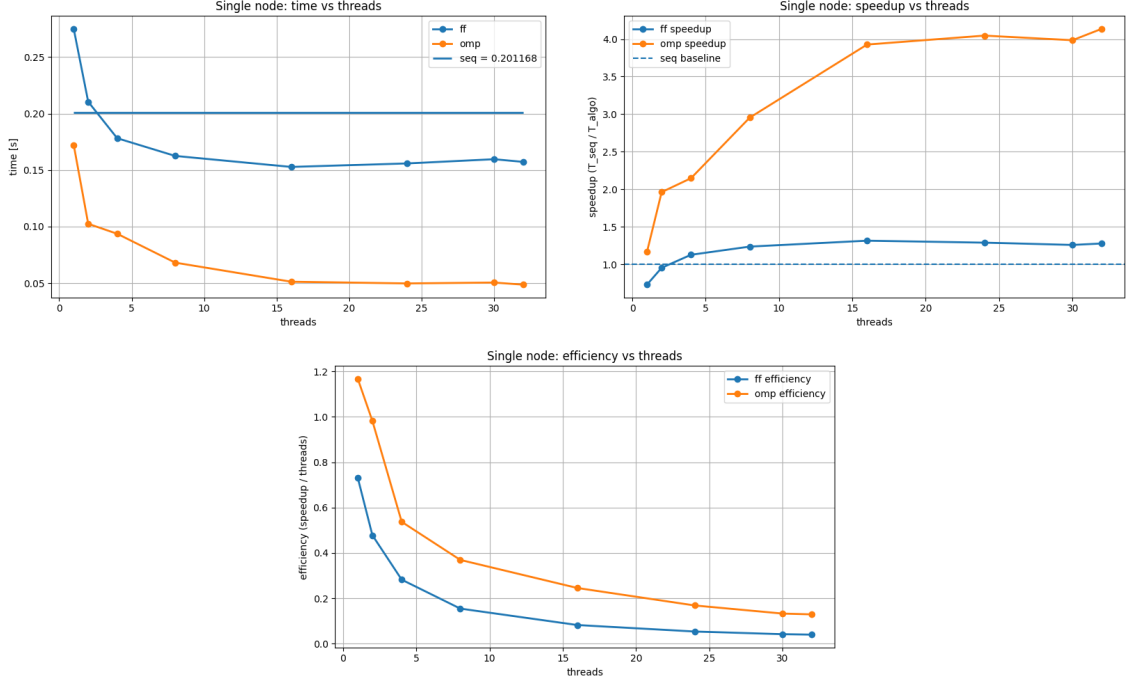


Figure 2: Single node test scenario B

is likely due to the overhead associated with dynamic task creation and scheduling in the recursive mergesort tree.

In Scenario B (smaller dataset but larger payload), the relative cost of the sorting phase decreases and the overall execution time becomes more influenced by memory access patterns and I/O effects. As a result, the scalability is reduced and the speedup curves flatten earlier.

Overall, the results confirm that parallelizing the index sorting phase significantly reduces execution time, while the achievable scalability is ultimately limited by memory bandwidth and the sequential parts of the pipeline.

7.2 Hybrid version

For the hybrid version (MPI + OpenMP), both strong scaling and weak scaling experiments were conducted. In all cases, up to 8 nodes were used with 32 cores per node.

In the **strong scaling** experiments, the total problem size remains constant while the number of nodes increases: $N = 10^8$, $payload_{max} = 32$. The number of nodes, the number of MPI ranks per node, and the number of OpenMP threads per rank were varied. Each configuration was executed twice.

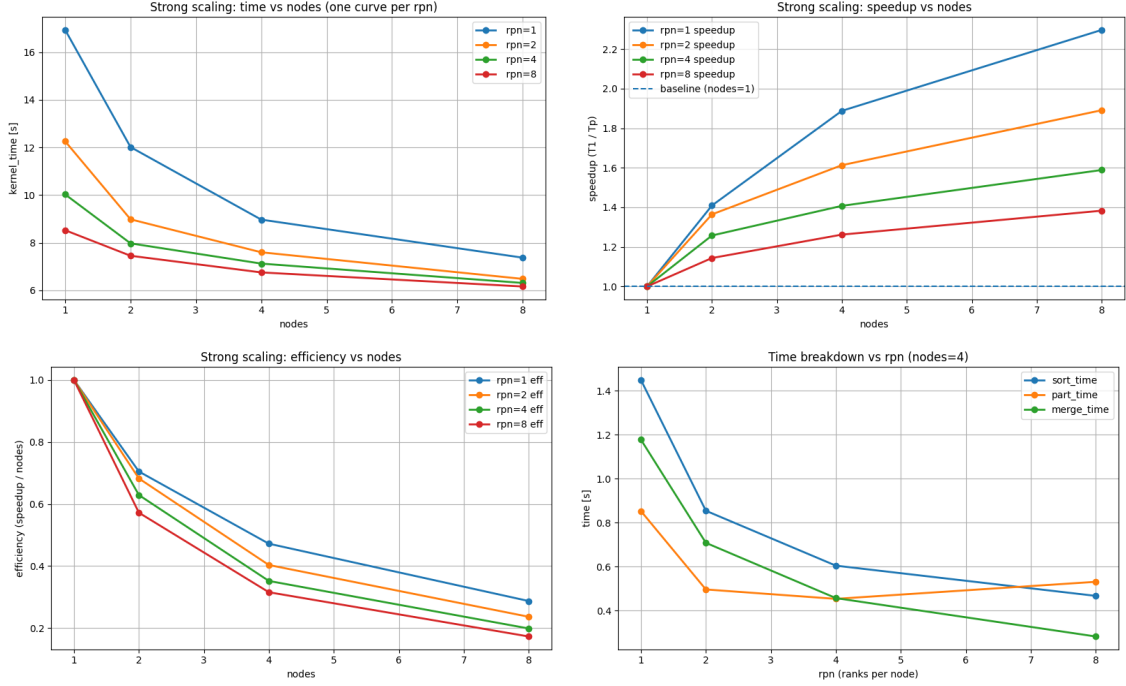


Figure 3: Strong scalability test

In the **weak scaling** experiments, the workload per process remains constant while the total problem size increases. The configuration used is $records_{per_rank} = 5 * 10^6$, $payload_{max} = 32$, while the number of nodes (1, 2, 4, 8) and the number of MPI ranks per node (1, 2, 4, 8) vary. The number of OpenMP threads per rank is computed so as to always utilize all the cores of the node.

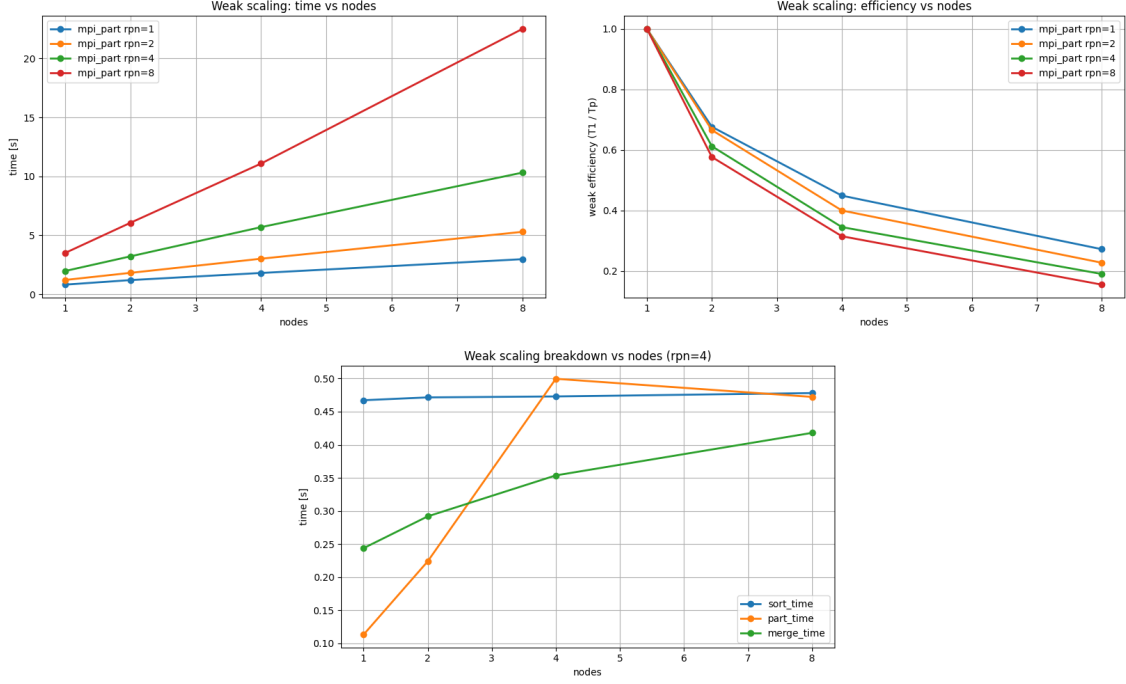


Figure 4: Weak scalability test

8 Cost Model

We denote by:

- N the total number of records
- d_{idx} the size of an IndexRec in bytes (≈ 24 bytes)
- P the number of MPI processes
- t the number of OpenMP threads per rank
- g the cost per byte of MPI communication
- l the MPI latency
- T_{read} the I/O cost per byte
- T_F the cost of a key comparison

The cost of the distributed version is modeled using the **BSP** framework. The cost of each superstep is given by the sum of the computational cost of the slowest process, the communication cost proportional to the volume of exchanged data, and the synchronization latency. In our algorithm, the MPI collectives (`MPI_Scatterv`, `MPI_Gatherv`, `MPI_Bcast`, `MPI_Alltoallv`) naturally act as barriers, making the BSP model appropriate. The total cost is the sum of the costs of the individual phases.

8.1 Index build

Rank 0 reads the file and builds the `global_idx` vector of N `IndexRec` elements:

$$T_{build} = B \cdot T_{read}$$

This phase is sequential and constitutes the main scalability bottleneck. $B = N \cdot r$ is the total file size in bytes, where r is the average size of a record. The distributed version correctly handles the case where the data does not fit into the total RAM of the cluster, since each rank works on a portion N/P of the index. However, the serial construction of the index on rank 0 requires that at least the global index ($N \cdot d_{idx}$ bytes) fits into the RAM of a single node. A fully out-of-core solution would require a distributed file read, for example by assigning each rank a portion of the file to read independently.

8.2 Initial distribution

Rank 0 sends N/P elements to each rank using `MPI_Scatterv`. This is preceded by an `MPI_Scatter` to distribute the counts, which is negligible.

$$T_{scatter} = l \log P + N \cdot d_{idx} \cdot g$$

. The total transmitted volume is $N \cdot d_{idx}$ bytes (the entire global index).

8.3 Local sorting with OpenMP

Each rank sorts its own chunk of N/P elements using t threads (task-based merge sort). All ranks operate in parallel, so the cost is determined by the slowest process:

$$T_{sort} = (N/P) \cdot \log(N/P) \cdot (T_F/t)$$

.

8.4 Sampling and splitter extraction

Each rank extracts $s = \min(1024, N/P)$ equally spaced samples. The samples are collected on rank 0 using `MPI_Gatherv` (volume $P \cdot 1024 \cdot 8$ bytes $\leq 8KB \cdot P$), sorted, and the $P - 1$ splitters are broadcast to all ranks. Since the volume is independent of N , this phase is negligible for large N .

$$T_{sample} \approx 0$$

8.5 Bucketization and exchange

Bucketization exploits the fact that `local_idx` is already sorted: for each of the $P - 1$ splitters, the boundary is found using `std::lower_bound` in $O(\log(N/P))$, for a total cost of

$$T_{bucket} = P \cdot \log(N/P) \cdot T_F$$

An `MPI_Alltoall` is performed to exchange the sendcounts:

$$T_{alltoallcounts} = l + P \cdot \text{sizeof}(int) \cdot g$$

`MPI_Alltoallv` redistributes the entire index: each rank sends a total of N/P elements and receives the same amount (assuming a good splitter balance):

$$T_{alltoall} = l \log P + N/P \cdot d_{idx} \cdot g$$

The overall cost of the phase is dominated by the latter:

$$T_{part} \approx l \log P + N/P \cdot d_{idx} \cdot g$$

8.6 Local k-way merge

Each rank receives P already sorted segments of average size N/P^2 each, for a total of N/P elements. The k-way merge with a min-heap of size P costs:

$$T_{merge} = \frac{N}{P} \cdot \log P \cdot T_F$$

8.7 Bottleneck analysis

By summing the individual costs, we obtain the total cost:

$$T_{tot} = B \cdot T_{read} + l \log P + N \cdot d_{idx} \cdot g + \frac{N}{P} \cdot \log \frac{N}{P} \cdot \frac{T_F}{t} + l \cdot \log P + \frac{N}{P} \cdot d_{idx} \cdot g + \frac{N}{P} \cdot \log P \cdot T_F$$

As discussed, T_{build} is the main bottleneck and limits strong scalability for large N . This is directly visible in the strong scaling plot (Figure 3, top left): the `kernel_time` does not decrease proportionally as the number of nodes increases because `T_build` remains constant independently of P . For the same reason, the maximum observed speedup is about 2.2x with 8 nodes (Figure 3, top right).

`T_sort` scales with $P \cdot t$. In the strong scaling time breakdown (Figure 3, bottom right), `sort_time` decreases as the number of ranks per node increases, confirming this proportionality.

`T_part` includes the `MPI_Alltoallv`, whose cost per rank is $N/P \cdot d_{idx} \cdot g$ — which decreases with P — but with an increasing latency $l \cdot \log P$. In the weak scaling experiment (Figure 4, bottom breakdown), `part_time` grows significantly with the number of nodes while keeping the workload per rank constant: this is consistent with the fact that latency increases and becomes the dominant component, even though the work per rank remains constant.

In `T_merge`, N/P decreases with P while $\log P$ grows slowly. In the breakdown (Figure 3 and Figure 4), `merge_time` remains the smallest and most stable component, confirming that the k-way merge does not represent a significant bottleneck.